

```
#!/usr/bin/env python3
```

```
-- coding: utf-8 --
```

```
"""
Python基础语法示例
展示了Python的核心语法特性
"""
```

```
import sys
import math
from typing import List, Dict, Optional
```

```
===== 1.
注释
=====
```

这是单行注释

```
"""
这是多行注释
可以用来写文档字符串
"""
```

变量和数据类型

```
print("=" * 50)
print("2. 变量和数据类型")
print("=" * 50)
```

数字类型

```
integer_var = 42          # 整数
float_var = 3.14159       # 浮点数
complex_var = 3 + 4j      # 复数
boolean_var = True        # 布尔值

print(f"整数: {integer_var}, 类型: {type(integer_var)}")
print(f"浮点数: {float_var}, 类型: {type(float_var)}")
print(f"复数: {complex_var}, 类型: {type(complex_var)}")
print(f"布尔值: {boolean_var}, 类型: {type(boolean_var)}")
```

字符串

```
string_var = "Hello, Python!"
multi_line_string = """这是
多行
字符串"""
print(f"字符串: {string_var}")
print(f"字符串长度: {len(string_var)}")
print(f"字符串切片: {string_var[0:5]}")
```

```
print(f"字符串大写: {string_var.upper()}")
```

列表 (可变序列)

```
my_list = [1, 2, 3, 4, 5]
my_list.append(6)
my_list.insert(0, 0)
print(f"列表: {my_list}")
print(f"列表切片: {my_list[2:5]}")
```

元组 (不可变序列)

```
my_tuple = (1, 2, 3, 4, 5)
print(f"元组: {my_tuple}")
```

字典 (键值对)

```
my_dict = {"name": "Alice", "age": 25, "city": "Beijing"}
my_dict["email"] = "alice@example.com"
print(f"字典: {my_dict}")
print(f"字典键: {my_dict.keys()}")
print(f"字典值: {my_dict.values()}")
```

集合 (无序不重复元素)

```
my_set = {1, 2, 3, 3, 4, 5} # 重复的3会被自动去重
my_set.add(6)
print(f"集合: {my_set}")
```

===== 3. 运算符

=====

```
print("\n" + "=" * 50)
print("3. 运算符")
print("=" * 50)
```

```
a, b = 10, 3
print(f"a = {a}, b = {b}")
print(f"加法: {a + b}")
print(f"减法: {a - b}")
print(f"乘法: {a * b}")
print(f"除法: {a / b}")
print(f"整除: {a // b}")
print(f"取余: {a % b}")
print(f"幂运算: {a ** b}")
```

比较运算符

```
print(f"a > b: {a > b}")
print(f"a == b: {a == b}")
print(f"a != b: {a != b}")
```

逻辑运算符

```
x, y = True, False
print(f"x and y: {x and y}")
print(f"x or y: {x or y}")
print(f"not x: {not x}")
```

===== 4. 控制流

=====

```
print("\n" + "=" * 50)
print("4. 控制流")
print("=" * 50)
```

if-elif-else 语句

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "D"
print(f"分数 {score} 对应的等级: {grade}")
```

for 循环

```
print("for循环示例:")  
for i in range(5):  
    print(f" i = {i}")
```

遍历列表

```
fruits = ["apple", "banana", "orange"]  
print("遍历列表:")  
for fruit in fruits:  
    print(f" {fruit}")
```

带索引的遍历

```
print("带索引的遍历:")  
for index, fruit in enumerate(fruits):  
    print(f" {index}: {fruit}")
```

while 循环

```
print("while循环示例:")  
count = 0  
while count < 3:  
    print(f" count = {count}")  
    count += 1
```

break 和 continue

```
print("break和continue示例:")
for i in range(10):
    if i == 3:
        continue # 跳过3
    if i == 7:
        break # 到达7时停止
    print(f" {i}", end=" ")
print() # 换行
```

===== 5.

函数

=====

```
print("\n" + "=" * 50)
print("5. 函数")
print("=" * 50)
```

基本函数定义

```
def greet(name: str) -> str:
    """简单的问候函数"""
    return f"Hello, {name}!"

print(greet("Python"))
```

带默认参数的函数

```
def power(base, exponent=2):  
    """计算幂，默认指数为2"""  
    return base ** exponent
```

```
print(f"2的3次方: {power(2, 3)}")  
print(f"2的平方: {power(2)}")
```

可变参数函数

```
def sum_all(*args):  
    """计算所有参数的和"""  
    return sum(args)
```

```
print(f"求和: {sum_all(1, 2, 3, 4, 5)}")
```

关键字参数函数

```
def person_info(name, **kwargs):  
    """打印个人信息"""  
    print(f"Name: {name}")  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")
```

```
person_info("Alice", age=25, city="Beijing", job="Engineer")
```


Lambda 函数

```
square = lambda x: x ** 2
print(f"Lambda平方: {square(5)}")
```

列表推导式

```
squares = [x ** 2 for x in range(10)]
print(f"列表推导式: {squares}")
```

===== 异常处理

=====
=====

```
print("\n" + "=" * 50)
print("6. 异常处理")
print("=" * 50)
```

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"除零错误: {e}")
except Exception as e:
    print(f"其他错误: {e}")
else:
    print(f"计算结果: {result}")
finally:
    print("无论是否发生异常，都会执行")
```

===== 7. 类和对象 =====

```
print("\n" + "=" * 50)
print("7. 类和对象")
print("=" * 50)
```

```
class Person:
    """人员类示例"""
```

```
# 类变量
species = "Homo sapiens"

def __init__(self, name: str, age: int):
    """构造函数"""
    self.name = name
    self._age = age # 私有属性（约定）

@property
def age(self):
    """属性getter"""
    return self._age

@age.setter
def age(self, value):
    """属性setter"""
    if value < 0:
        raise ValueError("年龄不能为负数")
    self._age = value

def introduce(self) -> str:
    """实例方法"""
```

```
        return f"我叫{self.name}, 今年{self._age}岁"

    @classmethod
    def get_species(cls):
        """类方法"""
        return cls.species

    @staticmethod
    def is_adult(age: int) -> bool:
        """静态方法"""
        return age >= 18
```

创建对象

```
person = Person("张三", 25)
print(person.introduce())
print(f"物种: {Person.get_species()}")
print(f"是否成年: {Person.is_adult(25)}")
```

继承

```
class Student(Person):
    """学生类，继承自Person"""
```

```
def __init__(self, name: str, age: int, student_id: str):
    super().__init__(name, age)
    self.student_id = student_id

def introduce(self) -> str:
    """重写父类方法"""
    return f"学生{super().introduce()}, 学号{self.student_id}"
```

```
student = Student("李四", 20, "2024001")
print(student.introduce())
```

===== 8. 模块和包

=====

```
print("\n" + "=" * 50)
print("8. 模块和包")
print("=" * 50)
```

导入模块

```
import math
from datetime import datetime

print(f"圆周率: {math.pi}")
print(f"平方根: {math.sqrt(16)}")
print(f"当前时间: {datetime.now()}")
```

文件操作

```
print("\n" + "=" * 50)
print("9. 文件操作")
print("=" * 50)
```

写入文件

```
with open("example.txt", "w", encoding="utf-8") as f:
    f.write("Hello, Python!\n")
    f.write("这是第二行\n")
```

读取文件

```
with open("example.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print("文件内容:")
    print(content)
```

10. 类型提示

```
print("\n" + "=" * 50)
print("10. 类型提示")
print("=" * 50)
```

```
def process_items(items: List[str]) -> Dict[str, int]:
    """处理列表，返回字典"""
    result = {}
    for item in items:
        result[item] = len(item)
    return result
```

```
items = ["apple", "banana", "orange"]
result = process_items(items)
print(f"处理结果: {result}")
```

=====

11. 常用内置函数

=====

```
print("\n" + "=" * 50)
print("11. 常用内置函数")
print("=" * 50)
```

```
numbers = [3, 1, 4, 1, 5, 9, 2, 6, 5]
print(f"原始列表: {numbers}")
print(f"长度: {len(numbers)}")
print(f"最大值: {max(numbers)}")
print(f"最小值: {min(numbers)}")
print(f"求和: {sum(numbers)}")
print(f"排序: {sorted(numbers)}")
print(f"去重: {list(set(numbers))}")
```

12. 生成器

```
print("\n" + "=" * 50)
print("12. 生成器")
print("=" * 50)
```

```
def fibonacci_generator(n):
    """斐波那契数列生成器"""
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b
```

```
print("斐波那契数列前10项:")
for num in fibonacci_generator(10):
    print(f" {num}", end=" ")
print() # 换行
```

13. 装饰器

```
print("\n" + "=" * 50)
print("13. 装饰器")
print("=" * 50)
```

```
def timer(func):  
    """计时装饰器"""  
    import time
```

```
    def wrapper(*args, **kwargs):  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()  
        print(f"{func.__name__} 执行时间: {end - start:.4f} 秒")  
        return result  
  
    return wrapper
```

@timer

```
def slow_function():  
    """模拟耗时操作"""  
    import time  
    time.sleep(1)  
    return "完成"
```

```
result = slow_function()  
print(f"结果: {result}")
```

=====

14. 上下文管理器

=====

```
print("\n" + "=" * 50)  
print("14. 上下文管理器")  
print("=" * 50)
```



```
class MyContext:
    """自定义上下文管理器"""
```

```
def __enter__(self):
    print("进入上下文")
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    print("退出上下文")
    if exc_type:
        print(f"发生异常: {exc_val}")
    return True # 抑制异常
```

```
with MyContext():
    print("在上下文中执行")
    # raise ValueError("测试异常")

print("程序继续执行")

print("\n" + "=" * 50)
print("Python基础语法示例完成！")
print("=" * 50)
```